LABORATORY ASSESSMENT

C PROGRAMMING — PRACTICAL QUESTION PAPER

Course Evaluation • Makeup Practical Examination

> **Instructions:** Write a complete C program for each question, including `#include` statements and a `main()` function. Keep code simple and add comments where needed. Questions progress systematically from fundamental concepts to advanced constructs across sessions.

SESSION 1 — PRINTF & VARIABLES

1. Print your name, class, and roll number using separate `printf()` statements.
2. Print a welcome message, e.g., "Welcome to C Programming".
3. Store your age in an `int` variable and print it.
4. Store the price of a pen in a variable and print "The price of a pen is 10 rupees".
5. Store a fruit's name (using a string/char array) and its price, then print both.
6. Print the number of students in a class using an `int` variable.

SESSION 2 — DATA TYPES & TYPE CASTING

7. Store the price of a chocolate as a `float` and print it.
8. Store the number of chairs as an `int` and the average weight of a chair as a `float`.
9. Convert an `int` age into a `float` and print both values.
10. Convert a `float` bus fare into an `int` and print the result.
11. Store a book's price as a `double` and print it with two decimal points.
12. Find the size of `int`, `float`, `char`, and `double` using the `sizeof()` operator.

SESSION 3 — OPERATORS & EXPRESSIONS

13. Add the price of two items and print the total cost.
14. Subtract a discount amount from the original price of a shirt.
15. Multiply the number of pens by the price of one pen to get the total cost.
16. Divide the total marks by the number of subjects to find the average.
17. Use the modulus operator to check if a given roll number is odd or even.
18. Increase a product's price by 10% using arithmetic operators.
19. Use increment (`++`) and decrement (`--`) operators to update a counter.

SESSION 4 — TAKING USER INPUT (SCANF)

20. Take the user's name as input and print "Hello, [name]!".
21. Take two numbers as input and print their sum.
22. Take the price of an item and quantity as input, then print the total bill.
23. Take a student's marks as input and print them back.
24. Take the user's age as input and print it.
25. Take a city name as input and print "You live in [city]".

SESSION 5 — CONDITIONAL STATEMENTS: IF-ELSE & SWITCH

26. Check if a number entered by the user is positive or negative.
27. Check if a student has passed or failed based on marks (pass mark = 40).
28. Check if a person is eligible to vote (age 18 or above).
29. Using `switch`, print the name of the day for a number entered (1 = Monday, 2 = Tuesday, etc.).
30. Check if a number is even or odd using `if-else`.
31. Check if a triangle is valid based on three side lengths entered by the user.
32. Using `if-else`, assign a grade (A, B, C, D) based on marks entered.

SESSION 6 — LOOPS: FOR, WHILE, DO-WHILE

33. Print numbers from 1 to 10 using a `for` loop.
34. Print the multiplication table of a number entered by the user.
35. Print all even numbers between 1 and 50 using a `while` loop.
36. Calculate the sum of numbers from 1 to n using a `for` loop.
37. Print "Good Morning" 5 times using a `do-while` loop.
38. Print a countdown from 10 to 1 using a loop.
39. Check if a number is a palindrome using a loop.

SESSION 7 — ARRAYS (1D & 2D)

40. Store the marks of 5 students in an array and print them.
41. Find the highest number in an array of 5 numbers.
42. Find the total and average of numbers stored in an array.
43. Store the names of 5 fruits in an array of strings and print each one.
44. Create a 2D array to store a 3x3 matrix and print it.
45. Count how many even numbers are present in an array.
46. Add two matrices using 2D arrays.

SESSION 8 — STRINGS (CHAR ARRAYS & STRING.H)

47. Take a person's name as input and print its length using `strlen()`.
48. Copy one string into another using `strcpy()`.
49. Compare two strings using `strcmp()`.
50. Concatenate two strings using `strcat()`.
51. Convert a given sentence to uppercase using a loop.
52. Reverse a given word using a loop or `strrev()`.
53. Check if a given word is a palindrome.

SESSION 9 — FUNCTIONS

54. Write a function to add two numbers and call it from `main()`.
55. Write a function that finds the square of a number.
56. Write a function to check if a number is prime.
57. Write a function `greet()` that prints "Hello, welcome!" and call it.
58. Write a function that takes a person's age and tells whether they are a minor or adult.
59. Write a function to find the factorial of a number.

SESSION 10 — RECURSION

60. Write a recursive function to find the factorial of a number.
61. Write a recursive function to print numbers from 1 to n.
62. Write a recursive function to find the sum of digits of a number.
63. Write a recursive function to calculate the Fibonacci series up to n terms.
64. Write a recursive function to reverse a given number.

SESSION 11 — POINTERS

65. Declare a pointer to an `int` variable and print the value using the pointer.
66. Write a program to swap two numbers using pointers.
67. Use a pointer to access and print elements of an array.
68. Write a function that uses a pointer parameter to double a number's value (call by reference).
69. Print the address of a variable using the `&` operator.

SESSION 12 — STRUCTURES & UNIONS

70. Create a structure `Student` with fields `name`, `rollNumber`, and `marks`, then print the details.
71. Create a structure `Employee` and store details of 3 employees using an array of structures.
72. Create a structure `Book` with a nested structure `Price` containing `cost` and `discount`.
73. Create a union to store either an `int` or a `float` value and observe the memory sharing.
74. Write a program to compare the size of a structure and a union with the same members.

SESSION 13 — DYNAMIC MEMORY ALLOCATION

75. Use `malloc()` to dynamically allocate memory for an array of 5 integers and store values in it.
76. Use `calloc()` to allocate memory for an array and print the initialized values.
77. Use `realloc()` to resize a previously allocated array.
78. Write a program to free allocated memory using `free()` and explain why it's needed.

SESSION 14 — FILE HANDLING

79. Write a program to create a text file and write a message into it using `fprintf()`.
80. Write a program to read and display the content of a text file using `fscanf()`.
81. Write a program to append data to an existing file.
82. Write a program to count the number of lines in a text file.
83. Write a program to copy the content of one file into another.

SESSION 15 — BONUS: PREPROCESSOR DIRECTIVES & MACROS

84. Use `#define` to create a macro for the value of PI and use it to find the area of a circle.
85. Use `#define` to create a macro that finds the square of a number.
86. Use conditional compilation (`#ifdef`, `#endif`) to include or exclude a part of code.
87. Use `#include` to demonstrate the difference between a user-defined header file and a standard header file.

——— • END OF QUESTION PAPER • ———